

State of the Art in Neutrino Telescopes: Simulation and Angular Reconstruction for the KM3NeT-ARCA Detector

Mouad DERFOUFI

Supervised by : Albert ARNAULD
mouad.derfoufi@unistra.fr

UFR de Physique et Ingénierie

Université de Strasbourg

Année académique 2025-2026

Abstract

This report presents an overview of high-energy neutrino astronomy, culminating in the computational simulation and directional reconstruction of muon tracks for the KM3NeT neutrino telescope. Traditional electromagnetic astronomy, while foundational, is fundamentally limited at extreme cosmic distances and energies due to the absorption and scattering of photons. To observe the universe’s most violent phenomena, a multi-messenger approach utilizing the neutrino—a nearly massless, weakly interacting fundamental particle—is required.

The first half of this report establishes the theoretical framework, detailing the physics of the neutrino, the principles of neutrino astronomy, and the mechanics of detecting these elusive particles via Cherenkov radiation in deep underwater environments. This context introduces KM3NeT, the current generation of optical Cherenkov detectors in the Mediterranean Sea.

The second half of the report details the practical side of the internship: the development of a custom physics simulation written in Python. This program models the KM3NeT detector’s geometry, simulates the passage of muons, and mathematically applies the detector’s optical triggers. Finally, the raw 4D spatial and temporal detection data is processed using the LineFit algorithm—an analytical, linear regression method—to reconstruct the original muon trajectory. The accuracy and limitations of this first-guess reconstruction method are evaluated based on the median angular error of the simulated events.

Contents

Introduction	5
1 High energy neutrino astrophysics	6
1.1 Traditional Astronomy and its Limits	6
1.2 The Neutrino	7
1.2.1 Neutrino Mass	7
1.2.2 Neutrino Sources	8
1.2.3 Neutrino Detection	9
1.2.4 Cherenkov Radiation	10
1.3 Pioneering Neutrino Telescopes	10
2 KM3NeT	12
2.1 ARCA and ORCA	12
2.2 DOMS	13
3 SIMULATION	14
3.1 Initialization	14
3.1.1 Muon Generation	15
3.2 Generating The Detector's Geometry	16
3.2.1 DU layout (2D Grid)	16
3.2.2 Vertical Extension (3D)	17
3.3 DOCA filter	17
3.4 Photon generation	19
3.4.1 Modeling the Cherenkov Cone	19
3.5 Physics engine	20
4 RECONSTRUCTION	22
4.1 Trajectory Reconstruction: The LineFit Algorithm	22
4.1.1 Introduction to Trajectory Reconstruction	22
4.1.2 The LineFit Mathematical Model	22
4.1.3 Performance Evaluation and Angular Resolution	23
5 Results / Analysis	24
5.1 Results and Analysis	24
5.1.1 Trigger Efficiency and Reconstruction Success	24

5.1.2	Angular Resolution and Algorithmic Limitations	24
5.1.3	Visual Analysis of Reconstruction Performance	25
	Conclusions	26
	Bibliography	27
	A APPENDIX	28
A.1	Python Source Code	28
A.2	The Simulation	28
A.3	LineFit Reconstruction	35

Introduction

While Galileo’s telescope expanded our cosmic horizons, astronomy remained fundamentally bound to the narrow band of visible light for centuries. The 20th century shattered this limitation, opening the entire electromagnetic spectrum to human observation. Astronomy became entirely dependent on the photon as its universal messenger. However, as astrophysicists pushed to observe the universe’s most extreme, high-energy events, they encountered a fundamental physical barrier.

At the highest energy scales—such as those generated by Active Galactic Nuclei and supernova remnants—the universe effectively becomes opaque. High-energy photons are highly susceptible to absorption; they interact with intergalactic dust and background radiation, losing their energy long before reaching Earth. Furthermore, while high-energy cosmic rays are routinely detected, their electrical charge causes their trajectories to be scrambled by intergalactic magnetic fields. To look deeper into the extreme universe, astronomy required a new messenger.

The high-energy neutrino provides the ultimate solution. As a nearly massless, electrically neutral particle that interacts only via the weak nuclear force, it travels through dense stellar cores and magnetic fields unhindered. Its trajectory points in a perfect line directly back to its violent source. Yet, this extraordinary penetrating power makes neutrinos exceptionally difficult to detect. Catching them requires massive, kilometer-scale volumes of ultra-transparent mediums like deep seawater. When a neutrino occasionally collides with an atomic nucleus in the water, it produces a secondary charged particle (such as a muon) that travels faster than the local speed of light. This generates a faint flash of blue Cherenkov radiation, captured by sensitive photomultiplier tubes. The KM3NeT project, currently under construction in the Mediterranean Sea, represents the next generation of these monumental detectors.

This report details the theoretical physics of neutrino astronomy and the development of a computational algorithm to simulate and reconstruct these interactions. Chapter 1 reviews high-energy astrophysics and Cherenkov detection principles. Chapter 2 details the architecture of the KM3NeT telescope. Finally, Chapters 3, 4, and 5 present the practical methodology of this internship: a Python simulation of the detector’s geometry, the mathematical implementation of a LineFit algorithm to reconstruct 4D muon trajectories, and a statistical analysis of the algorithm’s accuracy.

Chapter 1

High energy neutrino astrophysics

1.1 Traditional Astronomy and its Limits

For centuries, our understanding of the cosmos was built entirely upon the observation of visible light. The advent of modern astronomy expanded this horizon to encompass the entire electromagnetic (EM) spectrum. By deploying telescopes both on Earth and in orbit, astrophysicists can now observe the universe through radio waves, infrared, optical, ultraviolet, X-rays, and gamma rays. However, as researchers attempt to probe the absolute highest-energy phenomena—such as Active Galactic Nuclei and gamma-ray bursts—traditional photon-based astronomy encounters severe physical limitations. At extreme energy scales (typically in the TeV to PeV ranges), the universe becomes highly opaque to photons. When ultra-high-energy gamma rays travel across cosmological distances, they interact with pervasive background radiation, primarily the Cosmic Microwave Background (CMB). This interaction results in electron-positron pair production[1]:

$$\gamma + \gamma_{CMB} \rightarrow e^+ + e^- \quad (1.1)$$

Due to this, the highest-energy photons are annihilated long before they can reach Earth. This prevents astrophysicists from using the EM spectrum to directly observe distant, extreme cosmic accelerators. In addition to photons, Earth detects high-energy cosmic rays. However, because these particles are electrically charged, they are subject to the Lorentz force. As they travel through space, their trajectories are continuously deflected by galactic and intergalactic magnetic fields. By the time a cosmic ray reaches a detector on Earth, its directional information is completely scrambled, making it impossible to trace the particle back to its source. Therefore, probing the extreme universe requires a fundamentally different messenger. To carry intact directional and energetic information across cosmological distances, this messenger must satisfy two physical criteria: it must be electrically neutral to travel in a straight line through magnetic fields, and it must be weakly interacting to pass unhindered through dense matter and background radiation. The high-energy neutrino perfectly fulfills these criteria, making it the ideal candidate for high-energy multi-messenger astronomy.

1.2 The Neutrino

In 1930, Wolfgang Pauli postulated the existence of an "undetected particle" to explain how beta (β) decay could conserve energy, momentum, and angular momentum (spin). Shortly after, Enrico Fermi incorporated Pauli's particle into the first formal theory of weak interactions. To distinguish it from the newly discovered heavy neutron, Edoardo Amaldi coined the term **Neutrino**[2] .

Its existence was finally proven experimentally on July 20, 1956, by Clyde Cowan and Frederick Reines, a monumental discovery that later earned them the 1995 Nobel Prize in Physics.

Within the Standard Model of particle physics, the neutrino is classified as a lepton. It is a fundamental fermion with a spin (1/2) and carries no electrical charge. Because it is electrically neutral and lacks a color charge, the neutrino is completely unaffected by both the electromagnetic force and the strong nuclear force. It interacts with other matter exclusively through gravity and the weak nuclear force. Because the weak force has an incredibly short range, the neutrino possesses a vanishingly small interaction cross-section. This allows it to pass completely unhindered through planets, dense stellar cores, and vast cosmological distances.

Neutrinos come in three distinct "flavors" associated with their charged lepton partners: the electron neutrino (ν_e), the muon neutrino (ν_μ), and the tau neutrino (ν_τ).

1.2.1 Neutrino Mass

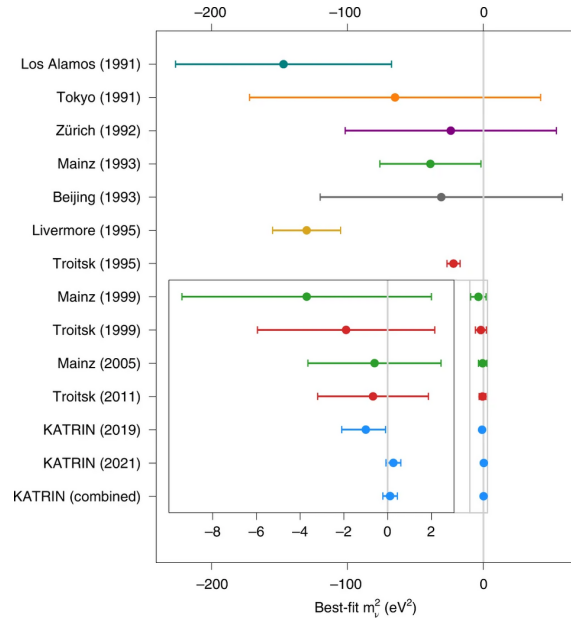


Figure 1.1: Timeline of electron neutrino mass measurements by different experiments.

For decades, the Standard Model assumed that all neutrinos were completely massless and traveled at exactly the speed of light. However, the groundbreaking discovery of neutrino oscillation—a quantum mechanical phenomenon where a neutrino changes its flavor as it travels through space- changed this view, and according to quantum mechanics,

for a particle to oscillate between states, it must possess a non-zero mass.

The Most Recent Mass Estimates[3] [2][1] :

$$\begin{cases} m_e < 0.45eV/c^2 \\ m_\mu < 190keV/c^2 \\ m_\tau < 18.2MeV/c^2 \end{cases} \quad (1.2)$$

1.2.2 Neutrino Sources

The production of high-energy astrophysical neutrinos requires extreme cosmic accelerators. In these environments, accelerated protons (cosmic rays) collide with ambient gas or dense radiation fields. These hadronic collisions produce unstable mesons—primarily charged pions (π^+ and π^-)—which rapidly decay to produce high-energy neutrinos via the following reaction chain [1]:

$$\begin{cases} \pi^+ \rightarrow \mu^+ + \nu_\mu \\ \mu^+ \rightarrow e^+ + \nu_e + \bar{\nu}_\mu \end{cases} \quad (1.3)$$

these high-energy collisions can be traced back to a few primary cosmic sources:

Active Galactic Nuclei (AGN): Supermassive black holes that accelerate particles along massive, relativistic jets.

Transient Explosions: The expanding shockwaves of Supernova Remnants and Gamma-Ray Bursts (GRBs), which provide the extreme magnetic fields required for particle acceleration.

While extreme cosmic accelerators produce the universe's highest-energy neutrinos, the vast majority of neutrinos passing through the Earth originate much closer, in the core of our Sun. The sun generates its energy through nuclear fusion, this process is dominated by the proton-proton (p-p) chain reaction. The initial and most crucial step of this chain involves two protons fusing to create deuterium (^2H), releasing a positron (e^+) and an electron neutrino (ν_e) to conserve lepton number [2]:

$$p + p \rightarrow ^2\text{H} + e^+ + \nu_e \quad (1.4)$$

While the Sun dominates the low-energy neutrino spectrum, the Earth's own atmosphere acts as a massive generator of high-energy neutrinos.

When standard galactic cosmic rays continuously bombard the Earth, they collide with nitrogen and oxygen nuclei in the upper atmosphere. These collisions trigger extensive "air showers" that produce identical pion decay chains, raining millions of atmospheric neutrinos and atmospheric muons

Source	Origin & Mechanism	Energy Range
Solar Neutrinos	Nuclear fusion (p-p chain) occurring in the Sun's dense core.	[0.1 MeV – 15 MeV]
Atmospheric Neutrinos	Galactic cosmic rays colliding with nitrogen/oxygen in Earth's upper atmosphere.	[100 MeV – 100 TeV]
Supernova Remnants	Expanding shockwaves from dying stars accelerating galactic cosmic rays.	[1 TeV – 1 PeV]
Gamma-Ray Bursts (GRBs)	Transient cosmological explosions (e.g., collapsing massive stars or merging neutron stars)	[100 TeV – 10 PeV]
Active Galactic Nuclei (AGN)	Relativistic jets powered by supermassive black holes actively consuming matter	[1 PeV – 1 EeV]

Table 1.1: Table summarizing the last section

1.2.3 Neutrino Detection

Because neutrinos carry no electrical charge and interact exclusively via the weak nuclear force, they cannot be observed directly. Instead, neutrino telescopes must detect them indirectly by observing the secondary particles produced when a neutrino occasionally collides with matter. To maximize the probability of these rare interactions, detectors require massive target volumes, utilizing cubic kilometers of dense, transparent mediums such as deep-sea water or glacial ice.

Neutrinos Interact with matter through **Neutral Currents (NC)** via Z^0 Boson or **Charged Currents(CC)** via $W^\pm$ Bosons , (CC) interactions produce leptons of the same flavor as the neutrino (e, μ, τ) followed with a hadronic shower , while (NC) interactions produce just the hadronic shower [4].

$$\begin{cases} \nu_l + N \rightarrow l^- + X & (\text{CC}) \\ \bar{\nu}_l + N \rightarrow l^+ + X & (\text{CC}) \\ \nu_l(\bar{\nu}_l) + N \rightarrow \nu_l(\bar{\nu}_l) + X & (\text{NC}) \end{cases} \quad (1.5)$$

The half life and travel distance of the interaction products (e, μ, τ) have different traces in the detector , for the (τ) its very short half life restrict the distance it travels though the detector leaving a point like trace ,as for the electron it scatters very quickly, leaving us with the muon (μ) ,a high-energy one can travel in a nearly perfect straight line through

kilometers of seawater, more than enough of a trace to reconstruct its trajectory.

1.2.4 Cherenkov Radiation

To actually observe this muon trajectory, deep-sea telescopes rely on the phenomenon of Cherenkov radiation, when light passes through a transparent medium like seawater, its phase velocity is reduced by the medium's refractive index ($n \approx 1.35$). Because the muon is created with immense relativistic energy, its physical velocity (v) exceeds the local speed of light in the water (c/n).

Much like a supersonic jet creating a sonic boom, the muon creates an electromagnetic shockwave as it forces its way through the water. It continuously emits a faint cone of blue and ultraviolet photons. This Cherenkov light is emitted at a characteristic angle (θ_C) relative to the muon's trajectory, defined by the equation [5]:

$$\cos(\theta_C) = \frac{1}{\beta n} \approx 0.74 \text{ for } \beta \approx 1 \quad (1.6)$$

This gives us: $\theta_C \approx 42^\circ$

By suspending an 3D array of highly sensitive optical sensors (Photomultiplier Tubes) in the darkness of the deep ocean- which acts as a shield against noise (cosmic rays and all particles coming from above) while earth blocks everything from below except neutrinos, meaning that a detection of an upward going muon implies the it's a byproduct of a neutrino with the matter below the detector-, physicists can record the exact arrival times and spatial coordinates of these Cherenkov photons. By mathematically analyzing this 4D geometric data (x, y, z, and time), reconstruction algorithms can perfectly trace the 42-degree light cone back to its source, revealing the precise trajectory of the muon and mapping the high-energy neutrino sky.

1.3 Pioneering Neutrino Telescopes

After the Cowan-Reines experiment, a century long engineering quest began to instrument natural bodies of water and ice. The foundational blueprint for this endeavor was established in the 1970s with the DUMAND (Deep Underwater Muon and Neutrino Detector) project. Proposed for the deep ocean off the coast of Hawaii, DUMAND was ultimately canceled before completion due to extreme deep-sea engineering challenges. However, it cemented the theoretical paradigm of using massive, naturally occurring Cherenkov arrays.

The first successful implementation of this underwater concept occurred in the 1990s with the Lake Baikal Neutrino Telescope (NT200) in Russia. By deploying optical strings in deep freshwater, the Baikal collaboration proved that underwater neutrino trajectory reconstruction was practically feasible.

Simultaneously, physicists began exploring deep glacial ice as an alternative detection medium. The AMANDA project at the South Pole successfully detected atmospheric neutrinos, paving the way for its monumental successor: IceCube. Completed in 2010,

IceCube instrumented a full cubic kilometer of Antarctic ice. In 2013, IceCube made history by discovering the first definitive flux of high-energy astrophysical neutrinos, officially inaugurating the era of multi-messenger neutrino astronomy.

However, while ice detectors like IceCube offer massive target volumes, optical scattering within glacial ice fundamentally limits their angular resolution (the precision with which they can point back to a cosmic source). To achieve the precise pointing accuracy required to identify individual galactic accelerators, astronomers returned to the deep sea.

The ANTARES telescope, completed in 2008 off the southern coast of France, served as the pioneering deep-sea Mediterranean detector. ANTARES demonstrated that the minimal light scattering of deep seawater offers superior optical properties for precise muon track reconstruction compared to ice. Operating for over a decade, the technological triumphs and engineering lessons learned from ANTARES serve as the direct, foundational stepping stone for its next-generation successor: the KM3NeT project [6][7][8].

Chapter 2

KM3NeT

As the successor of the ANTARES project, the Cubic Kilometre Neutrino Telescope (KM3NeT) represents the current generation of deep-sea Cherenkov detectors. Located in the deepest regions of the Mediterranean Sea, KM3NeT is designed to exploit the exceptional optical properties of deep seawater to reconstruct particle tracks with unprecedented angular resolution. However, KM3NeT is not a single structure; it is a distributed research infrastructure composed of two distinct detector arrays, each optimized for a specific physics objective and energy regime [8][6].

2.1 ARCA and ORCA

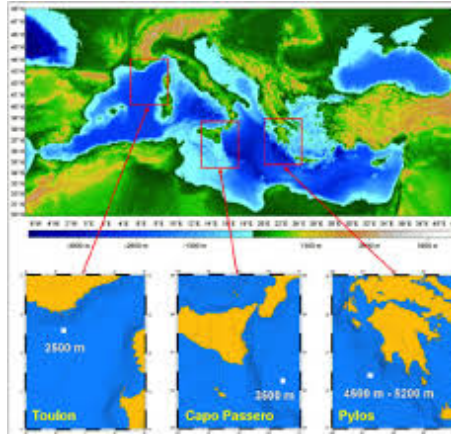


Figure 2.1: Geographical locations and operational depths of the KM3NeT sites in the Mediterranean Sea, including ORCA off the coast of Toulon and ARCA near Capo Passero and the Greece site under construction currently

the KM3NeT collaboration operates two separate sites: ARCA and ORCA. While both sites utilize identical foundational hardware and deployment technologies, their geometric configurations are vastly different.

ARCA (Astroparticle Research with Cosmics in the Abyss): Located off the coast of Capo Passero, Italy, at a depth of approximately 3,500 meters, ARCA is the high-energy observatory of the project. It is optimized to detect neutrinos in the TeV to PeV range. Its primary objective is to identify high-energy cosmic neutrinos and map extragalactic

astrophysical sources. Because high-energy neutrino interactions produce incredibly long muon tracks, ARCA's 115 lines (or detection unit DU) are widely spaced (90m) to maximize the total target volume, each holding 18 detection optical modules (DOMs) each DOM housing 31 photomultiplier tube (PMT), ultimately aiming to instrument over a full cubic kilometer of [6].

ORCA (Oscillation Research with Cosmics in the Abyss): Situated off the coast of Toulon, France, at a depth of 2,450 meters with a more the 115 DUs spaced only by 9m, ORCA is optimized for lower-energy atmospheric neutrinos, typically in the 1 GeV to 100 GeV range. Its primary scientific goal is to study neutrino oscillations to determine the elusive neutrino mass hierarchy. Because lower-energy interactions produce much shorter particle tracks, ORCA features a highly dense geometric configuration, with optical modules packed closely together to ensure these brief tracks trigger enough sensors for accurate reconstruction.

2.2 DOMS

The fundamental detection unit of the KM3NeT telescope is the Digital Optical Module (DOM). Because these modules must operate at depths exceeding 2,400 meters, the outer shell consists of a highly transparent, 17-inch pressure-resistant glass sphere designed to withstand extreme hydrostatic pressure while allowing maximum transmission of Cherenkov photons.

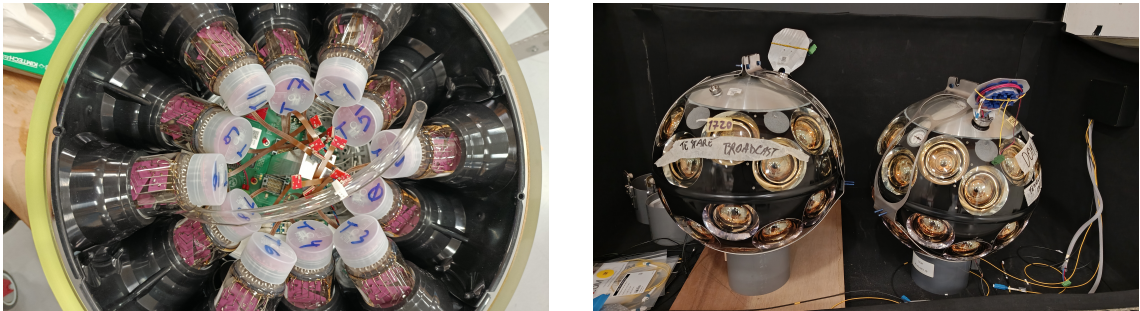


Figure 2.2: Pictures we took in our visit to the assembly lab in IPHC strasbourg ,The KM3NeT Digital Optical Module (DOM). Left: Internal view showing the 31 PMTs. Right: Fully assembled DOM.

[4]

As seen in the unsealed module during lab assembly, the PMTs are held in place by a 3D-printed support structure and are wired directly into a central processing board , additionally, each DOM contains its own internal compass, tiltmeter, and LED calibration beacon. This ensures that even as the flexible detector lines sway in the deep ocean currents, the exact 3D orientation and timing calibration of every single PMT is constantly monitored [6].

Chapter 3

SIMULATION

Before applying trajectory reconstruction algorithms to real, raw data from the Mediterranean Sea, it is standard practice to develop and test these mathematical models in a controlled, simulated environment. Simulating the detector allows physicists to generate ideal particle tracks with known parameters. By feeding this simulated data into a reconstruction algorithm and comparing the algorithm's output to the known input, the accuracy and efficiency of the code can be rigorously evaluated.

This chapter details the development of a Python simulation designed to model the geometric architecture of the KM3NeT detector, simulate the passage of highly relativistic muons, and generate the resulting 4D Cherenkov hit data.

3.1 Initialization

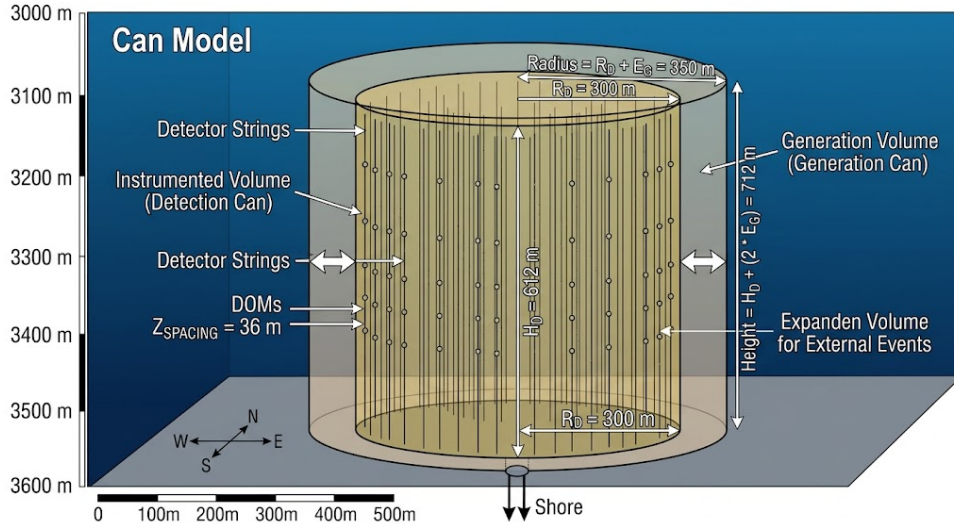


Figure 3.1: Visual representation of the simulation volumes. The inner cylinder (Instrumented Volume) is the physical KM3NeT detection units. The expanded outer cylinder (Generation Volume) defines the boundary where simulated high-energy muons are created.

Instrumented Volume (Detection Can): To accurately simulate muon events and their resulting Cherenkov light, we must mathematically define two distinct volumes in

the deep-sea water column, which are best visualized as nested cylinders (or "cans"). Instrumented Volume (Detection Can) This volume represents the region of water physically instrumented by the detector lines (strings). For your simulated KM3NeT-ARCA configuration, this is defined by: Radius ($\mathbf{R}_{\text{det}} = \mathbf{585m}$) (derived from the layout of 115 Detection Units with $\mathbf{90m}$ spacing). Height ($\mathbf{H}_{\text{det}} = \mathbf{612m}$) (set by $\mathbf{18}$ DOMs spaced $\mathbf{36}$ meters vertically). This volume encapsulates all the sensors that will "see" the Cherenkov light.

Generation Volume (Generation Can): This is an artificially expanded volume used to create the high-energy muon. It encompasses the Instrumented Volume and extends further outwards in all directions, with ($\mathbf{R}_{\text{gen}} = \mathbf{625m}$) and ($\mathbf{H}_{\text{gen}} = \mathbf{812m}$) which is ($\mathbf{H}_{\text{det}} + \mathbf{100m}$) from top and bottom.

3.1.1 Muon Generation

To produce a statistically random sample of particles, we generate N muons, assigning each a starting position and a directional vector.

1. Position: To ensure the muons originate exclusively within the boundaries of the Generation Volume, their initial starting coordinates (x_0, y_0, z_0) are generated using a cylindrical coordinate system. For each muon, the algorithm randomly and uniformly generates three independent variables in cylindrical coordinates:

$$\begin{aligned} 0 &\leq \rho \leq R_{\text{gen}} \\ 0 &\leq \alpha < 2\pi \\ z_{\text{min}} &\leq z \leq z_{\text{max}} \end{aligned} \tag{3.1}$$

These random cylindrical parameters are then transformed into standard 3D Cartesian coordinates to determine the precise origin point of the particle:

$$\begin{cases} x_0 = \rho \cos(\alpha) \\ y_0 = \rho \sin(\alpha) \\ z_0 = z \end{cases} \tag{3.2}$$

2. Direction(Trajectory):

Because a highly relativistic muon travels in a practically straight line, its trajectory is modeled as a 3D unit vector. The direction is initially calculated in a local spherical coordinate system by randomly sampling a zenith angle ($\theta \in [0, \pi]$) and an azimuthal angle ($\phi \in [0, 2\pi]$). This yields a local directional vector(muon frame):

$$\vec{d}_{\text{local}} = \begin{pmatrix} \sin(\theta) \cos(\phi) \\ \sin(\theta) \sin(\phi) \\ \cos(\theta) \end{pmatrix} \tag{3.3}$$

Finally, to map this trajectory correctly into the global coordinate space of the detector array, the local vector undergoes a rotation based on the particle's spatial origin angle (α).

By applying the corresponding rotation matrix, we obtain the absolute global directional vector ($\vec{d}_{global}$) for the simulated muon:

$$\vec{d}_{global} = R_z(\alpha)\vec{d}_{local} = \begin{pmatrix} -\cos(\alpha) & \sin(\alpha) & 0 \\ -\sin(\alpha) & -\cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \sin(\theta) \cos(\phi) \\ \sin(\theta) \sin(\phi) \\ \cos(\theta) \end{pmatrix} \quad (3.4)$$

and with this we have set the path for the muon with the starting position (x_0, y_0, z_0) and a direction of $(d_{x-global}, d_{y-global}, d_{z-global})$.

3.2 Generating The Detector's Geometry

With the simulation boundaries established, the physical architecture of the KM3NeT array must be manually reconstructed within the Detection Volume. This process is done in two steps: defining the 2D layout of the **115 line (DUs)** on the seabed, and extending them vertically into 3D space [9].

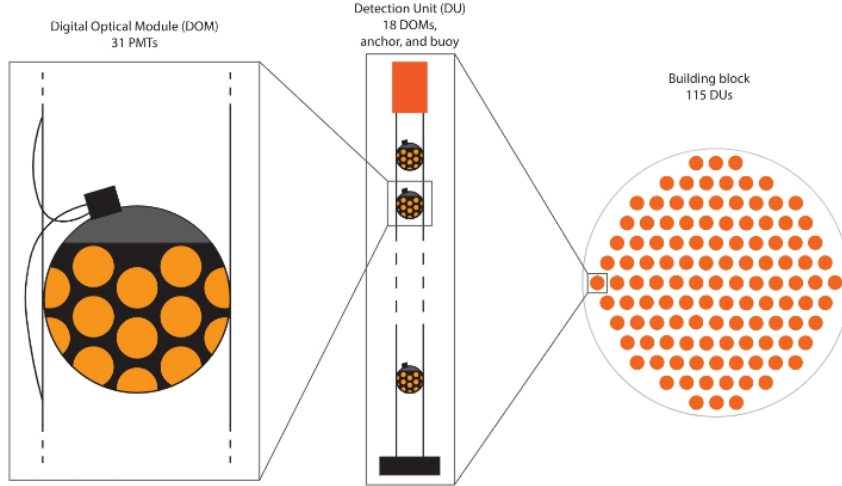


Figure 3.2: The layout of the Detector

3.2.1 DU layout (2D Grid)

The ARCA detector lines (Detection Units or DUs) are not arranged in a simple square matrix, as shown above but rather in a dense, triangulated grid that naturally forms a hexagonal lattice. To generate this structure, the simulation uses two mathematical basis vectors separated by an angle of $\pi/3$ (60°):

$$\vec{v}_1 = \begin{bmatrix} d \\ 0 \end{bmatrix}, \quad \vec{v}_2 = \begin{bmatrix} d \cos(\frac{\pi}{3}) \\ d \sin(\frac{\pi}{3}) \end{bmatrix} \quad (3.5)$$

Where $d = 90.0$ m represents the spacing between the lines. By taking linear combinations of these basis vectors using integer multipliers ($i, j \in [-6, 6]$), the simulation generates an initial oversized planar grid:

$$\vec{P}_{i,j} = i\vec{v}_1 + j\vec{v}_2 \quad (3.6)$$

This geometric expansion creates a structure consisting of roughly 6 to 7 concentric rings of DUs radiating outward from the central origin, producing a total of 127 anchoring points. However, the KM3NeT-ARCA building block is designed with exactly 115 strings. To accurately model its distinct, roughly circular footprint, this hexagonal grid is spatially filtered.

The distance from the origin $(0,0)$ is calculated for every generated point. The algorithm then systematically deletes the outermost DUs from the borders of the grid. By sorting the array and retaining only the 115 coordinates closest to the center, the simulation yields a precise 115×2 matrix representing the absolute (x,y) anchoring points of the telescope.

3.2.2 Vertical Extension (3D)

To populate the instrumented volume with DOMs, these 115 spatial anchoring points are extended vertically along the z -axis.

Each of the simulated Detection Units is equipped with 18 DOMs. The lowest DOM on each string is positioned near the bottom boundary of the Detection Volume ($z_{min} = -306.0$ m), with each following DOM spaced vertically by $\Delta z = 36.0$ m. The discrete z -coordinate for the k -th DOM on any given string is determined mathematically by:

$$z_k = z_{min} + k \cdot \Delta z \quad \text{for } k \in \{0, 1, 2, \dots, 17\} \quad (3.7)$$

By appending these 18 calculated z -values to each of the established (x,y) seabed coordinates, the 2D grid is extruded into a fully three-dimensional array. This geometric operation yields the final simulated detector architecture: a 2070×3 matrix.

$$\text{Detector Matrix} = \begin{pmatrix} x_1 & y_1 & z_{1,0} \\ x_1 & y_1 & z_{1,1} \\ \vdots & \vdots & \vdots \\ x_{115} & y_{115} & z_{115,17} \end{pmatrix} \quad (3.8)$$

Each row in this comprehensive matrix contains the (x,y,z) spatial coordinates of a single DOM, establishing the precise coordinates required for the physics engine to calculate Cherenkov photon intersections.

3.3 DOCA filter

Simulating the emission of tens of thousands of Cherenkov photons for every single optical module across the entire instrumented array would be very computationally intensive. The vast majority of the 2,070 simulated DOMs will be physically too far from the muon's trajectory to detect any light due to water absorption. To drastically optimize the physics engine, a strict spatial filter is applied before any photon propagation is calculated.

This optimization utilizes the **Distance of Closest Approach (DOCA)**, defined as the shortest orthogonal distance between a specific point in space (the DOM) and the muon trajectory.

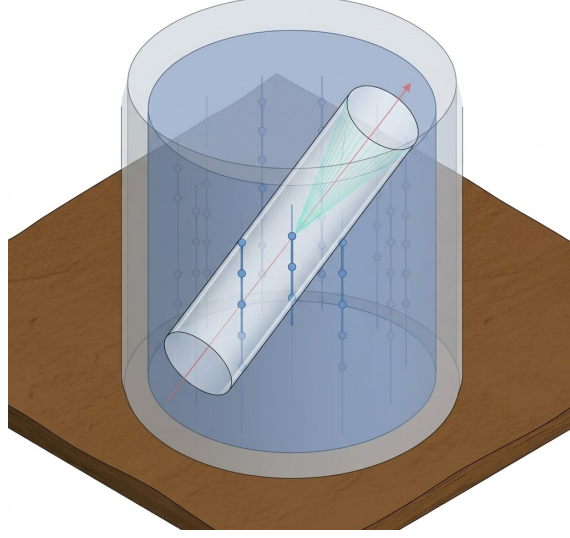


Figure 3.3: A representation of the filter shown as the cylinder surrounding the muon trajectory

Let $\vec{P}_0$ represent the generation coordinates of the muon, and $\vec{u}$ represent its global directional unit vector ($\vec{d}_{global}$). For any given DOM located at coordinates $\vec{P}_{DOM}$, we can define a position vector $\vec{L}$ connecting the muon's starting point to the DOM:

$$\vec{L} = \vec{P}_{DOM} - \vec{P}_0 \quad (3.9)$$

Because $\vec{u}$ is a normalized unit vector, the exact perpendicular distance from the DOM to the muon's infinite flight path can be rapidly computed using the magnitude of the cross product of these two vectors:

$$d_{DOCA} = ||\vec{L} \times \vec{u}|| \quad (3.10)$$

This calculation is performed using arrays to instantaneously evaluate the DOCA for all 2,070 DOMs simultaneously. A strict distance threshold of $d_{max} = 40.15$ m is then applied. The threshold ensures that only DOMs within a reasonable range of the Cherenkov cone are considered.

The typical attenuation length of Cherenkov photons in the deep-sea site is approximately $L_{max} \approx 60$ m. Because the photons are emitted at a characteristic Cherenkov angle of $\theta_C = \arccos(0.74) \approx 42.2^\circ$, the maximum lateral distance perpendicular to the muon's trajectory can be obtained using:

$$d_{max} = L_{max} \cdot \sin(\theta_C) = 60 \cdot \sin(42.2^\circ) \approx 40.3 \text{ m} \quad (3.11)$$

As the muon is continuously emitting a cone of light this will trace a cylinder with a radius if $r_{DOCA} = 40.15$ m, meaning that any DOM that fails this geometric condition

$d_{DOCA} \leq 40.15$ m is completely masked out of the subsequent simulation steps. This spatial filter aggressively reduces the active computational matrix from 2,070 coordinates down to a minimal subset of "triggered" DOMs, allowing the step-by-step physical simulation to run highly efficiently without sacrificing scientific accuracy.

$$\text{Trigger Condition: } d_{DOCA} \leq 40.155 \text{ m} \quad (3.12)$$

3.4 Photon generation

Photon Generation and Coordinate Transformation

For every discrete spatial step (Δs) the muon takes along its trajectory, the simulation must generate a corresponding burst of Cherenkov photons. In seawater, a highly relativistic muon emits approximately $N \approx 34,500$ photons per meter in the (300-600 nm)[5].

Photon Yield via the Frank-Tamm Formula

To accurately model Cherenkov light, the simulation must determine the exact number of photons emitted per unit length of the muon's trajectory. This is governed by the Frank-Tamm formula.

For a highly relativistic muon ($\beta \approx 1$) passing through seawater ($n = 1.35$), we can calculate the Cherenkov yield (N_o) within a specific wavelength window $[\lambda_1, \lambda_2]$ using:

$$\frac{dN_o}{dl} = 2\pi\alpha \left[1 - \frac{1}{n^2} \right] \frac{\lambda_2 - \lambda_1}{\lambda_1 \lambda_2} \quad (3.13)$$

Here, α is the fine-structure constant ($\approx 1/137$). The integration limits match the optical sensitivity of the DOM's Photomultiplier Tubes (PMTs), which operate efficiently in the ultraviolet-to-blue spectrum ($\lambda_1 = 300$ nm, $\lambda_2 = 600$ nm) [5].

Evaluating this equation yields approximately 345 Cherenkov photons per centimeter.

$$\frac{dN_o}{dl} = 34,500 \text{ photons/meter} \quad (3.14)$$

Consequently, as the simulated muon steps forward, the physics engine generates exactly $34,500 \cdot \Delta s$ photon directional vectors for every spatial step size Δs .

3.4.1 Modeling the Cherenkov Cone

Each emitted photon is initially generated in a local spherical coordinate system where the muon's trajectory acts as the z -axis. The photon's directional vector is defined by the constant Cherenkov zenith angle ($\theta_C = 42.2^\circ$) and a uniformly sampled random azimuthal angle ($\phi \in [0, 2\pi]$). This establishes the local photon trajectory as a column vector:

$$\vec{d}_{local} = \begin{pmatrix} \sin(\theta_C) \cos(\phi) \\ \sin(\theta_C) \sin(\phi) \\ \cos(\theta_C) \end{pmatrix} \quad (3.15)$$

To accurately align this local Cherenkov cone with the muon's true trajectory in the

detector's global space, a rotation matrix (R) is constructed using the **Gram-Schmidt orthogonalization** process.

The muon's global directional unit vector ($\vec{u}$) is assigned as the new basis vector $\vec{w}$. To prevent collinearity singularities, an arbitrary, non-parallel vector $\vec{a}$ is selected to calculate a perpendicular vector $\vec{u}'$. Finally, the cross product $\vec{v}' = \vec{w} \times \vec{u}'$ completes the right-handed orthonormal basis:

$$R = \begin{pmatrix} \vec{u}' & \vec{v}' & \vec{w} \end{pmatrix} \quad (3.16)$$

By applying this rotation matrix to the locally generated vectors, the physics engine dynamically projects the 3D Cherenkov cone along the correct physical flight path of the muon:

$$\vec{d}_{global} = R \cdot \vec{d}_{local} \quad (3.17)$$

3.5 Physics engine

With the active array minimized by the DOCA filter, the simulation models the particle's progression iteratively. The muon is stepped forward along its trajectory in discrete spatial intervals (Δs). At each step, a burst of Cherenkov photon vectors ($\vec{\gamma}_i$) is emitted.

1. Geometric Hit Detection

For every surviving DOM located at $\vec{P}_{DOM}$, the engine evaluates geometric intersections with the emitted photon rays. Let $\vec{L}$ represent the spatial vector connecting the muon's current position to the DOM:

$$\vec{L} = \vec{P}_{DOM} - \vec{P}(s) \quad (3.18)$$

To determine if a photon strikes the sensor, the scalar projection of $\vec{L}$ onto the photon's directional unit vector is computed via the dot product: $d_{\parallel} = \vec{L} \cdot \vec{\gamma}_i$. The condition $d_{\parallel} > 0$ strictly ensures the photon is propagating *towards* the DOM, rather than away from it.

Applying the Pythagorean theorem, the perpendicular distance ($d_{\perp}$) between the photon's infinite ray path and the center of the DOM is calculated:

$$d_{\perp}^2 = ||\vec{L}||^2 - d_{\parallel}^2 \quad (3.19)$$

A successful detection, or "hit," is recorded exclusively if this perpendicular distance is less than or equal to the physical radius of the DOM ($d_{\perp} \leq R_{DOM} = 0.21$ m).

2. Temporal Calibration

The defining metric for trajectory reconstruction is the precise arrival time of these hits. Because the simulated particle and its emitted light travel at different relativistic speeds, the absolute arrival time (t) of a photon at a DOM is the sum of two distinct propagation times:

$$t = \frac{s}{c} + \frac{d_{\parallel}}{v_{water}} \quad (3.20)$$

Where s is the total track distance the muon traveled until emitting the photon, c is the speed of light in a vacuum, $d_{\parallel}$ is the distance the photon traveled through the medium, and v_{water} is the phase velocity of light in seawater.

The final output of the simulation is a discrete $N \times 4$ spatial-temporal matrix, where each row contains the x, y, z coordinates of a triggered DOM alongside its relative hit time t . This highly accurate 4D dataset mathematically mirrors the raw output of the physical KM3NeT telescope, perfectly setting the stage for algorithmic trajectory reconstruction.

$$M_{event} = \begin{bmatrix} x_1 & y_1 & z_1 & t_1 \\ x_2 & y_2 & z_2 & t_2 \\ \vdots & \vdots & \vdots & \vdots \\ x_N & y_N & z_N & t_N \end{bmatrix} \quad (3.21)$$

Chapter 4

RECONSTRUCTION

4.1 Trajectory Reconstruction: The LineFit Algorithm

4.1.1 Introduction to Trajectory Reconstruction

The primary objective of a neutrino telescope is to reconstruct the original flight path of an incident particle based exclusively on the spatial-temporal data of the detected Cherenkov light. To achieve this, the 4D event matrix M_{event} generated by the physics engine is processed through a "first-guess" reconstruction algorithm. For this the **LineFit** algorithm was implemented.

4.1.2 The LineFit Mathematical Model

The LineFit algorithm operates on a simplified premise: it ignores the complex 42.2° Cherenkov emission cone and instead assumes that the recorded optical hits propagate along a 1D straight line through the detector space at a constant velocity $\vec{v}$.

Mathematically, this assumes the position of a triggered DOM ($\vec{r}_i$) is linearly proportional to its trigger time (t_i) [10]:

$$\vec{r}_i \approx \vec{r}_0 + \vec{v} \cdot t_i \quad (4.1)$$

Where $\vec{r}_0 = (x_0, y_0, z_0)$ represents an arbitrary starting position and $\vec{v} = (v_x, v_y, v_z)$ represents the fitted velocity vector. To extract these parameters, the algorithm performs three independent linear least-squares regressions. The components of the spatial coordinates (x, y, z) are isolated and plotted against the temporal array (t) using:

$$\begin{aligned} x(t) &= x_0 + v_x t \\ y(t) &= y_0 + v_y t \\ z(t) &= z_0 + v_z t \end{aligned} \quad (4.2)$$

Once the components of the velocity vector $\vec{v}$ are extracted from the slope of the plots, the vector is normalized to yield the final reconstructed directional unit vector ($\vec{d}_{rec}$):

$$\vec{d}_{rec} = \frac{\vec{v}}{||\vec{v}||} \quad (4.3)$$

All of this is done using the Numpy prebuilt function `Polyfit()`.

4.1.3 Performance Evaluation and Angular Resolution

To rigorously evaluate the accuracy of the LineFit algorithm and the underlying simulation, the reconstructed trajectory ($\vec{d}_{rec}$) is mathematically compared to the (true), generated muon trajectory ($\vec{d}_{true}$) that was initially generated in the Generation Volume.

Because both variables are normalized unit vectors, the deviation between them is calculated using the dot product. The angular error ($\Delta\theta$), representing the absolute reconstruction error in radians, is derived directly from the inverse cosine of this product:

$$\Delta\theta = \arccos(\vec{d}_{rec} \cdot \vec{d}_{true}) \quad (4.4)$$

This calculation yields a discrete angular error for every successfully reconstructed event in the simulation. By aggregating these individual errors, summary statistics (such as the mean and median angular deviations) are extracted, providing a quantitative metric for the overall resolving power of the simulated KM3NeT architecture.

Chapter 5

Results / Analysis

5.1 Results and Analysis

To evaluate the performance of the simulation and the LineFit reconstruction algorithm, a computational run of $N = 5000$ highly relativistic muons was executed within the Generation Volume, the run took around 13 hours.

5.1.1 Trigger Efficiency and Reconstruction Success

Of the 5000 generated particles, 3585 muons (71.7%) successfully intersected the KM3NeT-ARCA array and triggered at least one Digital Optical Module (DOM). The remaining particles exclusively traversed the outer boundaries of the detection volume without depositing detectable light.

The $N \times 4$ temporal-spatial matrices for the 3585 triggered events were processed through the LineFit algorithm. LineFit successfully reconstructed a trajectory vector ($\vec{d}_{rec}$) for 3361 events, yielding an exceptional algorithmic success rate of 93.7%. The 224 failed events correspond to "grazing" muons that produced only a single DOM hit, providing insufficient geometric data ($N < 2$) for a linear regression fit.

5.1.2 Angular Resolution and Algorithmic Limitations

For every successfully reconstructed event, the absolute angular deviation ($\Delta\theta$) between the true generated trajectory ($\vec{d}_{true}$) and the LineFit projection ($\vec{d}_{rec}$) was calculated. The analysis of these 3361 events yielded the following statistics:

- **Median Angular Error:** 4.31°
- **Mean Angular Error:** 9.47°
- **Minimum Error (Best Fit):** 0.009°
- **Maximum Error (Worst Fit):** 130.83°

The highly accurate median error of 4.31° mathematically validates the physics engine; a vast majority of the reconstructed tracks aligned closely with their true physical paths.

However, the disparity between the median (4.31°) and the mean (9.47°), alongside extreme outlier maximums ($> 100^\circ$), exposes the limitations of the LineFit "first-guess" approach. By aggressively assuming a 1D straight-line topology, LineFit fundamentally ignores the 42.2° Cherenkov emission angle. If a muon produces asymmetrical hits on a single vertical detector string, the linear regression inevitably fits the track directly *along* that string, causing severe angular distortions. To push the angular resolution below 1° , these LineFit parameters must be utilized as initial seed values for advanced Maximum Likelihood Estimation (MLE) algorithms that explicitly model the Cherenkov cone [10].

5.1.3 Visual Analysis of Reconstruction Performance

To visually contextualize the performance of the LineFit algorithm and the underlying physics engine, the reconstruction data was mapped into two key distributions, presented in Figure 5.1.

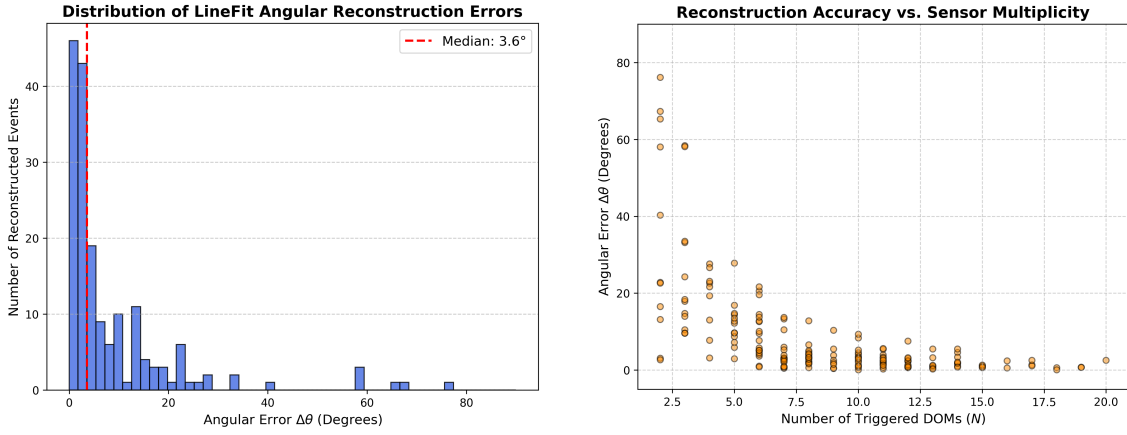


Figure 5.1: (Left) Distribution of angular errors showing a peak in the sub- 5° range with a median of 3.6° . (Right) Scatter distribution demonstrating the convergence of angular error as a function of triggered DOM multiplicity.

Sadly the breakdown of the hits data for the 5000 muon simulations was lost and overwritten (breakdown of how many unique DOMs hit by how many photons) this data was done only using 250 muons but it is still a decent enough sample to showcase the performance our work.

Correlation: Distance vs. Photon Intensity

The simulation accurately reproduces the $1/r$ geometric thinning of the Cherenkov shock-wave. As seen in the breakdown of typical events within the 250-muon sample, DOMs situated close to the path of the muon (< 5 m) recorded massive hits exceeding 350 photons, whereas distant DOMs near the edge of the physical light cone (> 30 m) recorded faint hits of fewer than 40 photons. This inverse relationship confirms that the engine correctly models the spatial density of the light cone as it expands through the water medium.

Conclusions

In this internship study we have demonstrated the feasibility of modeling high-energy muon events with both physical rigor and computational efficiency. Throughout this project, we moved from the macro-scale hardware constraints—characterizing the hexagonal string geometry and the optical sensitivity of the (DOMs) at 3,500 meters depth—to the micro-scale physics of light propagation. The integration of the Frank-Tamm formula and a Gram-Schmidt orthonormal basis allowed for the successful generation of the 42.2° Cherenkov wavefront, while the development of a Distance of Closest Approach (DOCA) filter proved essential in managing the high-density photon flux (34,500 photons/m) that characterizes these relativistic events. The final validation phase proved that the simulated $N \times 4$ spatial-temporal datasets (x, y, z, t) effectively mirror the trace produced by the physical detector. With a success rate of 93.7% and a median angular resolution of 4.31° , the results confirm that the simulation provides a high-fidelity environment for trajectory estimation. While the limitations of the 1D LineFit algorithm were identified in outlier cases, the project concludes that the physics engine is reliable. This simulation now serves as a scalable foundation for more advanced studies, such as the implementation of Maximum Likelihood Estimation (MLE) or neural-network-based track reconstruction, ultimately contributing to the predictive power required for the next generation of neutrino astronomy.

Bibliography

- [1] Franck BERNARD. *Caractérisation des performances d'un télescope sous-marin à neutrinos pour la détection de cascades contenues dans le cadre du projet ANTARES*. PhD thesis, Université de Strasbourg, 2000.
- [2] Wikipedia contributors. Neutrino — Wikipedia, the free encyclopedia. <https://en.wikipedia.org/wiki/Neutrino>, 2024. Accessed: 2026-05-08.
- [3] Doriance STUBERT. *Contribution à l'expérience ANTARES : évaluation de site et étude de la sensibilité à la détection de neutrinos*. PhD thesis, Université de Strasbourg, 2002.
- [4] Sophie FERRY. *Étude de la production de neutrinos associés aux Sursauts Gamma dans le modèle du boulet de canon. Possibilité d'observation de ces neutrinos par le détecteur ANTARES, et étude du bruit de fond optique enregistré par le prototype d'un secteur de ligne*. PhD thesis, Université de Strasbourg, 2004.
- [5] Jürgen Brunner. Cherenkov light from he electromagnetic and hadronic showers. *ANTARES internal note*, page 63, 2002.
- [6] Wikipedia contributors. Cherenkov radiation — Wikipedia, the free encyclopedia. <https://en.wikipedia.org/wiki/KM3NeT>, 2024. Accessed: 2026-05-08.
- [7] Wikipedia contributors. Cherenkov radiation — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/List_of_neutrino_experiments, 2024. Accessed: 2026-05-08.
- [8] *A Deep Sea Telescope for High Energy Neutrinos*. 1999.
- [9] KM3NeT Collaboration km3net-pc@ km3net. de, S Aiello, A Albert, M Alshamsi, S Alves Garre, Z Aly, A Ambrosone, F Ameli, M Andre, E Androutsou, et al. Astronomy potential of km3net/arca. *The European Physical Journal C*, 84(9):885, 2024.
- [10] J. Ahrens and others (AMANDA Collaboration). First-guess track reconstruction for amanda. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 524(1-3):169–194, 2004.

Appendix A

APPENDIX

A.1 Python Source Code

This appendix contains the core Python scripts developed for the KM3NeT-ARCA building block simulation and the subsequent data reconstruction.

A.2 The Simulation

```
1 import numpy as np # type: ignore
2 import time
3
4
5 #1.CONSTANTS
6 D_det = 1170.0
7 D_gen = D_det + 80.0
8
9 H_det = 612.0
10 H_gen = H_det+200
11
12 R_gen = D_gen / 2.0
13 gen_z_max = H_gen / 2.0
14 gen_z_min = -H_gen / 2.0
15
16 D_SPACING = 90.0
17 N_DU = 115
18
19 N_DOMS = 18
20 Z_SPACING = 36.0
21
22 # Consts for Photon generation & Time
23 THETA_C = np.radians(42.0) # Cherenkov angle
24 PHOTONS_PER_METER = 34500
25 R_DOM = 0.21 # DOM radius
26
27 C_VACUUM = 0.29979 # Speed of light in vacuum (meters per nanosecond)
28 N_WATER = 1.35 # Refractive index of Seawater
29 V_WATER = C_VACUUM / N_WATER # Speed of light in water
```

```

30
31
32 #2.MUON GENERATION
33 def generate_muons(N):
34     # POSITIONS
35     rho = np.random.uniform(0, R_gen, N)
36     alpha = np.random.uniform(0, 2 * np.pi, N)
37     z = np.random.uniform(gen_z_min, gen_z_max, N)
38
39     x = rho * np.cos(alpha)
40     y = rho * np.sin(alpha)
41
42     mu_position = np.column_stack((x, y, z))
43
44     #DIRECTIONS
45     phi = np.random.uniform(0, 2 * np.pi, N)
46     theta = np.random.uniform(0, np.pi, N)
47
48     dx_local = np.sin(theta) * np.cos(phi)
49     dy_local = np.sin(theta) * np.sin(phi)
50     dz_local = np.cos(theta)
51
52     # muon frame to detector frame
53     dx_global = -np.cos(alpha) * dx_local + np.sin(alpha) * dy_local
54     dy_global = -np.sin(alpha) * dx_local - np.cos(alpha) * dy_local
55     dz_global = dz_local
56
57     mu_direction = np.column_stack((dx_global, dy_global, dz_global))
58
59     return mu_position, mu_direction
60
61
62 #DU coordinates function (X,Y)
63 def generate_du_positions():
64     v1 = np.array([D_SPACING, 0.0])
65     v2 = np.array([D_SPACING * np.cos(np.pi/3), D_SPACING * np.sin(np.pi/3)
66 ])
67
68     #empty list
69     positions = []
70     for i in range(-6, 7):
71         for j in range(-6, 7):
72             pos = i * v1 + j * v2
73             positions.append(pos)
74
75     # list to array
76     positions = np.array(positions)
77     distances = np.linalg.norm(positions, axis=1)
78
79     # Filtering(127 DU to 115 DU)
80     closest_indices = np.argsort(distances)[:N_DU]
81     return positions[closest_indices]

```

```

81
82 #Extantion 3D
83 def build_3d_detector(du_positions):
84     dom_positions = []
85     z_start = -H_det / 2.0 # Starts exactly at -306.0
86
87     for du in du_positions:
88         x = du[0]
89         y = du[1]
90         for i in range(N_DOMS):
91             z = z_start + (i * Z_SPACING)
92             dom_positions.append([x, y, z])
93
94     return np.array(dom_positions)
95
96
97 # DETECTION FILTER
98 def filter_triggered_doms(mu_start, mu_dir, dom_positions, max_distance
99 =50.0):
100     # A is the mu starting point, u unit directional vectors , P DOM
101     dom_positions
102     AP_vectors = dom_positions - mu_start
103     cross_products = np.cross(AP_vectors, mu_dir)
104
105     doca_array = np.linalg.norm(cross_products, axis=1)
106
107     # 4. filter mask
108     triggered_mask = doca_array <= max_distance
109
110     # 5.the surviving DOMs and their exact distances
111     doms_left = dom_positions[triggered_mask]
112     distances_left = doca_array[triggered_mask]
113
114     return doms_left, distances_left
115
116 def get_rotation_matrix(w):
117     # as we already know the direction of the muon trajectory in the global
118     # frame we take it as the local z axis represented by w a unit vector
119     w = w / np.linalg.norm(w)
120
121     # Safety checks before Gram-Schmidt orthogonalisation , we are picking
122     # an arbitrary vector x or y but we dont want that vector to be the muon
123     # travelling direction
124     if abs(w[0]) > 0.9:
125         a = np.array([0.0, 1.0, 0.0])
126     else:
127         a = np.array([1.0, 0.0, 0.0])
128
129     # Gram-Schmidt
130     u = a - np.dot(a, w) * w
131     u = u / np.linalg.norm(u)

```

```

128     v = np.cross(w, u)
129
130     return np.column_stack((u, v, w))
131
132
133 def generate_photon_rays(mu_dir, n_photons):
134     #Generates photon direction vectors in the Cherenkov cone and rotates
    them to global
135
136     # 1. In the local frame (The cone)
137     phi = np.random.uniform(0, 2 * np.pi, n_photons)
138
139     # Spherical to cartesian
140     x_local = np.sin(THETA_C) * np.cos(phi)
141     y_local = np.sin(THETA_C) * np.sin(phi)
142     z_local = np.full(n_photons, np.cos(THETA_C))
143
144     local_dirs = np.column_stack((x_local, y_local, z_local))
145
146     # 2. Rotate to Global Muon Frame
147     R = get_rotation_matrix(mu_dir)
148     global_dirs = np.dot(local_dirs, R.T)
149
150     return global_dirs
151
152
153 def Physics_engine(mu_start, mu_dir, dom_positions, step_size=1.0,
    max_distance=40.0):
154
155     # Applying The FILTER
156     hit_doms, hit_distances = filter_triggered_doms(mu_start, mu_dir,
    dom_positions, max_distance=max_distance)
157
158     print(f"DOM Filter reduced 2070 DOMs to just {len(hit_doms)} DOMs!")
159
160     if len(hit_doms) == 0:
161         print(" -> Muon completely missed the detector. Skipping Phy engine
    !")
162         return None, None # <--- JUST ADD 'None, None' HERE!
163
164     # ENGINE
165     current_pos = np.copy(mu_start)
166
167     #array to keep track of the doms that detected a photon
168     dom_hit_counts = np.zeros(len(hit_doms))
169
170     #array to keep track of when the first photon hit each DOM
171     dom_first_hit_times = np.full(len(hit_doms), np.inf)
172
173     step_count = 0
174     start_time = time.time()
175

```

```

176 while True:
177     # Check boundaries
178     x, y, z = current_pos[0], current_pos[1], current_pos[2]
179     radius = np.sqrt(x**2 + y**2)
180     if radius > R_gen or z < gen_z_min or z > gen_z_max:
181         break
182
183     step_count += 1
184
185     # Generate the 34,500 photon direction arrows for this step
186     photon_dirs = generate_photon_rays(mu_dir, int(PHOTONS_PER_METER *
step_size))
187
188     # We use enumerate to keep track of the index i of the DOM we are
checking
189     for i, dom in enumerate(hit_doms):
190
191         #similar to the A and P right triangle we used for the filter L
is the hypotenious dom = P current pos = A
192         L = dom - current_pos
193
194         #photon-dom alignment to check if the photon is travelling in
the direction of dom or away from it (or also the adjesent side in the
right triangle)
195         ph_dom_align = np.dot(photon_dirs, L)
196
197         #condition
198         valid_rays = ph_dom_align > 0
199
200         # np.any checks the all the 34500 values(bool) in valid rays,
if at least one is true it returns true
201         if not np.any(valid_rays):
202             continue
203
204         L_sq = np.dot(L, L)
205         d2 = L_sq - (ph_dom_align[valid_rays]**2)
206
207         # Create a mask specifically for the hits
208         hit_mask = d2 <= R_DOM**2
209         hits_on_this_dom = np.sum(hit_mask)
210
211         if hits_on_this_dom > 0:
212             # Add the hits to this specific DOM's score
213             dom_hit_counts[i] += hits_on_this_dom
214
215             # Delay CALCULATIONS
216             #Muon travel time (distance = steps * size, speed = C)
217             muon_distance = step_count * step_size
218             t_muon = muon_distance / C_VACUUM
219
220             # 2. Photon travel time (distance = projection, speed =
V_WATER)

```



```

221         # We apply both masks to get only the distances of the
photons that hit!
222         photon_distances = ph_dom_align[valid_rays][hit_mask]
223         t_photon = photon_distances / V_WATER
224
225         # Total absolute physical time since simulation started
226         arrival_times = t_muon + t_photon
227
228         # 3. Update the scoreboard with the EARLIEST hit time for
this specific DOM
229         earliest_hit = np.min(arrival_times)
230         if earliest_hit < dom_first_hit_times[i]:
231             dom_first_hit_times[i] = earliest_hit
232
233         # Move muon forward
234         current_pos = current_pos + (mu_dir * step_size)
235
236     total_time = time.time() - start_time
237
238     # Statistics
239     total_hits = int(np.sum(dom_hit_counts))
240
241     # Count how many DOMs have a score greater than 0
242     unique_doms_hit = int(np.sum(dom_hit_counts > 0))
243
244     print(" Simulation Complete ")
245     print(f"Total distance travelled inside the volume: {step_count} m")
246     print(f"Total Photons detected : {total_hits}")
247     print(f"Total Unique DOMs Hit: {unique_doms_hit} (out of {len(hit_doms)}
} filtered DOMs)")
248     print(f"Total Time Taken to simulate this muon: {total_time:.2f}
seconds")
249
250     if unique_doms_hit > 0:
251         print("\nBreakdown of detections per DOM:")
252
253         # Find the very first hit time in the entire detector to act as our
t=0 reference point
254         first_detector_hit = np.min(dom_first_hit_times[dom_hit_counts >
0])
255
256         for i, counts in enumerate(dom_hit_counts):
257             if counts > 0:
258                 dist = hit_distances[i]
259                 relative_time = dom_first_hit_times[i] - first_detector_hit
260                 print(f" -> DOM {i+1} (Distance: {dist:.1f}m) recorded {int
(counts)} photons detected | Delay: {relative_time:.1f} ns")
261
262         #extracting qnd returning the data
263         hit_mask = dom_hit_counts > 0
264         final_hit_coords = hit_doms[hit_mask]

```

```

265     final_hit_times = dom_first_hit_times[hit_mask] -
        first_detector_hit
266
267     return final_hit_coords, final_hit_times
268
269     # If there were 0 hits, return None so the main loop knows to ignore it
270     return None, None
271
272 # EXECUTION & DATA EXPORT
273
274 N = int(input("Enter the number of simulated muons : "))
275
276 #Generate the Muons and the Detector
277 mu_position, mu_direction = generate_muons(N)
278 du_positions = generate_du_positions()
279 detector_3d = build_3d_detector(du_positions)
280
281 #Create lists to collect our successful data
282 all_hit_data_4d = []
283 valid_muon_starts = []
284 valid_muon_dirs = []
285 valid_event_ids = []
286
287 #Loop through EVERY generated muon
288 for i in range(N):
289     print("\n" + "="*50)
290     print(f" SIMULATING MUON #{i+1} OF {N}")
291     print("="*50)
292
293     current_start = mu_position[i]
294     current_dir = mu_direction[i]
295
296
297     hits, times = Physics_engine(current_start, current_dir, detector_3d,
        step_size=0.5)
298
299
300     if hits is not None:
301         # Merge X, Y, Z, and t into one 4D matrix
302         merged_4d_matrix = np.column_stack((hits, times))
303
304         all_hit_data_4d.append(merged_4d_matrix)
305         valid_muon_starts.append(current_start)
306         valid_muon_dirs.append(current_dir)
307         valid_event_ids.append(i)
308
309 # FILE SAVING
310 print("\n" + "="*50)
311 print(f"Simulation Complete! {len(valid_event_ids)} out of {N} muons
        produced hits.")
312 print("Saving dataset...")
313

```

```

314
315 np.savez('km3net_sim_data.npz',
316          event_ids=valid_event_ids,
317          hit_data=np.array(all_hit_data_4d, dtype=object),
318          true_starts=valid_muon_starts,
319          true_dirs=valid_muon_dirs)
320
321
322 print("SUCCESS: 4D Data [X, Y, Z, t] saved to 'km3net_sim_data.npz'")
323 print("*"*50)

```

Listing A.1: The main simulation script handling muon generation and Cherenkov propagation.

A.3 LineFit Reconstruction

```

1 import numpy as np # type: ignore
2
3 # 1. Define the LineFit function
4 def LineFit(hit_matrix):
5     if len(hit_matrix) < 2:
6         return None
7
8     x = hit_matrix[:, 0]
9     y = hit_matrix[:, 1]
10    z = hit_matrix[:, 2]
11    t = hit_matrix[:, 3]
12
13    vx, x0 = np.polyfit(t, x, 1)
14    vy, y0 = np.polyfit(t, y, 1)
15    vz, z0 = np.polyfit(t, z, 1)
16
17    velocity_vector = np.array([vx, vy, vz])
18    direction_vector = velocity_vector / np.linalg.norm(velocity_vector)
19
20    return direction_vector
21
22 # =====
23 # TESTING THE ALGORITHM OVER ALL EVENTS
24 # =====
25
26 # Loading the dataset
27 data = np.load('km3net_sim_data.npz', allow_pickle=True)
28
29 all_hit_data = data['hit_data']
30 true_dirs = data['true_dirs']
31
32 num_events = len(all_hit_data)
33 print("Loaded events:", num_events)
34
35 angular_errors = []
36 num_hits_list = []
37 failed_reconstructions = 0

```

```

38
39 # --- Open the text file to permanently save the output ---
40 with open('reconstruction_results.txt', 'w') as file:
41     file.write(f"Loaded events: {num_events}\n\n")
42
43     for i in range(num_events):
44         current_hits = all_hit_data[i]
45         actual_true_direction = true_dirs[i]
46
47         guessed_direction = LineFit(current_hits)
48
49         if guessed_direction is not None:
50             dot_prod = np.dot(guessed_direction, actual_true_direction)
51             dot_prod = np.clip(dot_prod, -1.0, 1.0)
52
53             error_angle_rad = np.arccos(dot_prod)
54             error_angle_deg = np.degrees(error_angle_rad)
55
56             angular_errors.append(error_angle_deg)
57             num_hits_list.append(len(current_hits))
58
59             # Format the string, print it to terminal, and write it to the
60             file
61             result_line = f"Muon {i+1} hits: {len(current_hits)} Error: {
62             error_angle_deg}\n"
63             print(result_line.strip())
64             file.write(result_line)
65         else:
66             failed_reconstructions += 1
67             fail_line = f"Muon {i+1} hits: {len(current_hits)} Failed\n"
68             print(fail_line.strip())
69             file.write(fail_line)
70
71 # --- Write the Summary Stats to the bottom of the file ---
72 file.write("\nSummary Stats:\n")
73 file.write(f"Total Processed: {num_events}\n")
74 file.write(f"Successful: {len(angular_errors)}\n")
75 file.write(f"Failed: {failed_reconstructions}\n")
76
77 if len(angular_errors) > 0:
78     file.write(f"Mean Error: {np.mean(angular_errors)}\n")
79     file.write(f"Median Error: {np.median(angular_errors)}\n")
80     file.write(f"Min Error: {np.min(angular_errors)}\n")
81     file.write(f"Max Error: {np.max(angular_errors)}\n")
82
83 # =====
84 # TERMINAL SUMMARY
85 # =====
86 print("\nSummary Stats:")
87 print("Total Processed:", num_events)
88 print("Successful:", len(angular_errors))
89 print("Failed:", failed_reconstructions)

```

```

88
89 if len(angular_errors) > 0:
90     print("Mean Error:", np.mean(angular_errors))
91     print("Median Error:", np.median(angular_errors))
92     print("Min Error:", np.min(angular_errors))
93     print("Max Error:", np.max(angular_errors))
94
95 print("\nSUCCESS: All data successfully saved to 'reconstruction_results.
    txt'")

```

Listing A.2: The reconstruction algorithm.